Compte rendu : TP Vue.js 3

Gestion d'un parc informatique

Nom : Diallo Rougayatou

1. Introduction

Ce TP consistait à réaliser une application Vue.js 3 permettant de consulter, ajouter, modifier et supprimer des ordinateurs. L'application comporte une page de liste et une page commune d'ajout et de modification. Les données sont conservées dans le navigateur grâce au `localStorage`, sans serveur ni base de données.

2. Les composants et fichiers

- `App.vue` : la page principale. Elle affiche l'en-tête et la page choisie par le router (`<RouterView />`). Elle enregistre aussi les 5 ordinateurs d'exemple au premier lancement.
- `router/index.js` : il définit les trois routes de l'application (`/ordinateurs`, `/ordinateurs/ajouter` et `/ordinateurs/modifier/:id`) avec Vue Router.
- `OrdinateurCard.vue` : un composant réutilisable qui affiche la carte d'un ordinateur. Il reçoit l'ordinateur grâce aux props, et prévient la page de la liste, grâce à emit, quand on clique sur « Supprimer ».
- `ListeOrdinateurs.vue` : la page de la liste. Elle affiche les cartes avec `v-for`, gère la suppression avec `confirm()`, la recherche, le tri et le compteur.
- `EditOrdinateur.vue` : la page du formulaire, utilisée pour l'ajout et la modification. Les champs sont liés aux variables avec `v-model`.

3. Les fonctionnalités

- Affichage des ordinateurs sous forme de cartes (photo, modèle, processeur, mémoire, disque, carte graphique).
- Ajout : le formulaire démarre vide. Le modèle et le processeur sont obligatoires, la mémoire et le disque doivent être strictement positifs. Si le formulaire est valide, on retourne à la liste.
- Modification : la même page que l'ajout. Elle sait dans quel mode elle est grâce à l'adresse : si elle contient un `id`, c'est une modification et les champs sont préremplis.
- Suppression : une confirmation est demandée avec `confirm()`, puis la liste est mise à jour immédiatement.
- Persistance : le tableau est sauvegardé dans le `localStorage` avec `JSON.stringify` et relu avec `JSON.parse`. Les exemples ne sont ajoutés qu'à la première utilisation.
- Bonus : recherche par modèle ou processeur (avec un `computed`, donc sans bouton), compteur d'ordinateurs et tri par capacité mémoire (croissant ou décroissant).

4. Notions utilisées

`ref`, `computed`, `v-for` avec `:key`, `v-if`, `v-model`, `@click`, `@submit.prevent`, props, emit, Vue Router et `localStorage`.

5. Difficultés rencontrées

- Les notions nouvelles. Le router, les props, emit et le `localStorage` n'avaient pas encore été vus. J'ai eu du mal à comprendre comment la carte et la page de la liste communiquent : les props vont de la page vers la carte, et emit va de la carte vers la page.
- Une erreur d'import. Vite affichait « Failed to resolve import » parce que le fichier `EditOrdinateur.vue` n'existait pas avec le bon nom dans le dossier `views`. Il fallait vérifier l'orthographe exacte du fichier.
- L'affichage des images. Avec le dossier `src/assets/images`, on ne peut pas écrire le chemin directement : il faut utiliser `new URL(..., import.meta.url)`. De plus, mes images ne s'affichaient pas à cause des anciens noms de fichiers conservés dans le `localStorage`.
- Comprendre le `localStorage`. Après une suppression, l'ordinateur ne revenait pas, même en relançant `npm run dev`, car les données sont gardées dans le navigateur. J'ai dû vider la sauvegarde depuis l'onglet Application des outils de développement pour retrouver les exemples.
- Le bouton « Choisir une image ». Une page web ne peut pas copier un fichier dans le dossier du projet. Le bouton remplit donc seulement le champ Photo avec le nom du fichier, qui doit aussi être placé dans `src/assets/images`.

6. Conclusion

Ce TP m'a permis de découvrir la navigation entre plusieurs pages, la communication entre composants et la sauvegarde de données dans le navigateur. L'application est fonctionnelle et respecte l'énoncé, bonus compris.